Nesne Yönelimli Programlama – Konu Anlatımı

Yazılım Mühendisliğinin Kapsamı

Slaytlar, ders kitabı (Schach 8e), ders videoları ve WhatsApp grubu temel alınmıştır

9 Kasım 2026

1 Yazılım Mühendisliğinin Kapsamı (Chapter 1)

Bu bölümde ne var, sınavda nasıl çıkıyor?

Chapter 1 kitabın giriş bölümüdür: *neden yazılım mühendisliğine ihtiyaç var*, *bakım (maintenance) neden bu kadar önemli*, *neden ayrı bir planlama / test / dokümantasyon fazı yok* ve *nesne yönelimli paradigma neden daha iyi* sorularının cevabıdır. Sınavda bu bölümden genellikle

- kısa tanım soruları ("What is regression fault / moving target / modern maintenance?"),
- test (çoktan seçmeli) soruları (bakım türleri, ayrı faz olmayan etkinlik, modern bakım tanımı) ve
- **etik kurallar** (Software Engineering Code of Ethics, 8 ilke) gelir.

Hoca dersi İngilizce anlatır; cevaplarda terimlerin **İngilizcesini** kullanın. Bu dosyada her başlığın sonunda sınava yazılabilecek kısa İngilizce cevap kalıpları verilmiştir.

1.1 Tarihsel boyut: yazılım krizi (Historical aspects)

- **1968 NATO Konferansı (Garmisch, Almanya):** "software engineering" terimi burada ortaya atıldı. Amaç **software crisis**'i (yazılım krizi) çözmekti.
- Yazılım krizinin üç belirtisi: yazılım **geç teslim edilir (late)**, **bütçeyi aşar (over budget)** ve **içinde kalıntı hatalar vardır (with residual faults)**.
- **Standish Group 2006 verisi:** projelerin yalnızca %35'i başarılı; %46'sı tamamlandı ama geç / bütçe üstü / eksik özellikli (challenged); %19'u iptal edildi ya da hiç kullanılmadı. Yani "üç projeden ancak biri başarılı".
- **Cutter Consortium 2002:** kurumların %78'i dava ile biten anlaşmazlık yaşamış. Bunların %67'sinde teslim edilen sistem söz verilen işlevi karşılamamış, %56'sında teslim tarihi defalarca kaymış, %45'inde hatalar sistemi kullanılamaz hâle getirmiş.
- Sonuç: kriz çözülmedi; kitap buna **software depression** (uzun süren, kötü seyirli) der.

Sınav cevabı kalıbı

Software crisis: software is delivered late, over budget and with residual faults. The term software engineering was coined at the 1968 NATO conference in Garmisch to solve this crisis.

1.2 Ekonomik boyut (Economic aspects)

Örnek: yeni kodlama yöntemi CM_{new}, eskisi CM_{old}'dan %10 daha hızlı. Kullanmalı mıyız?

- **Sağduyu cevabı:** Tabii ki!
- **Yazılım mühendisliği cevabı:** önce (1) personelin **eğitim maliyeti**, (2) yeni teknolojiye geçişin **etkisi** (yeni araçlar, ek personel, düzenin bozulması) ve (3) yöntemin **bakıma etkisi** düşünülmelidir.

Kodlama + birim testi toplam maliyetin yalnızca yaklaşık $0{,}34 \times 0{,}25 = \%8{,}5$'idir. Bu yüzden kodlamayı %10 hızlandırmak toplam maliyeti en fazla %0,85 düşürür. Buna karşılık bakım maliyetini %10 düşüren bir teknik toplam maliyeti $0{,}10 \times \%75 = \%7{,}5$ düşürür. **Asıl kazanç bakımdadır.**

1.3 Bakım boyutu (Maintenance aspects)

1.3.1 Yaşam döngüsü modeli ve klasik fazlar

- **Life-cycle model:** yazılım geliştirirken izlenecek adımların (fazların) teorik tarifi. "Ne yapılmalı".
- **Life cycle:** belirli bir ürün için *gerçekte* uygulanan adımlar.

Klasik (waterfall, 1970) modelin fazları ve her fazdaki etkinlikler:

Faz	Yapılanlar
1. Requirements	Kavramı keşfet (explore the concept), müşterinin gereksinimlerini ortaya çıkar (elicit).
2. Analysis (specification)	Gereksinimleri analiz et, **specification document** hazırla, **SPMP** (software project management plan) hazırla. "Ürün *ne* yapacak?"
3. Design	Mimari tasarım (architectural design: modüllere ayırma), ardından ayrıntılı tasarım (detailed design: her modülün veri yapısı ve algoritması). "Ürün bunu *nasıl* yapacak?"
4. Implementation	Kodlama, birim testi (unit testing), bütünleştirme (integration), kabul testi (acceptance testing).
5. Postdelivery maintenance	Corrective, perfective ve adaptive bakım.
6. Retirement	Ürünün işlevine artık ihtiyaç kalmayınca kullanımdan kaldırılması.

1.3.2 Üç bakım türü

Tür	Tanım	Örnek
Corrective (düzeltici)	Kalıntı hataların giderilmesi; spesifikasyon değişmez (*software repair*).	Faiz yanlış hesaplanıyor, düzeltilir.
Perfective (mükemmelleştirici)	Müşterinin ürünün etkinliğini artıracağını düşündüğü değişiklikler: yeni işlev, daha kısa yanıt süresi.	Rapora yeni sütun eklemek, sistemi hızlandırmak.
Adaptive (uyarlayıcı)	Ürünün çalıştığı **ortam** değişince yapılan değişiklikler: yeni donanım / işletim sistemi, yeni yasal düzenleme.	KDV oranı değişti; Teal Tractors'ta Kanada vergisi ve CAD para birimi.

Perfective ve adaptive birlikte **enhancement** (software update) diye anılır. "Reliable" bir bakım türü değildir (çıkmış test sorusu).

1.3.3 Klasik ve modern bakım tanımları

Klasik tanım (development-then-maintenance model): Ürün kurulduktan sonra yapılan her değişiklik bakımdır. Bu **temporal** (zamana bağlı) bir tanımdır: bir etkinliğin bakım mı geliştirme

mi olduğu *ne zaman yapıldığına* bağlıdır.

- *Sonuç 1:* Bir hata kurulumdan **bir gün sonra** bulunup düzeltilirse klasik bakım; aynı hata kurulumdan **bir gün önce** düzeltilirse klasik geliştirme sayılır.
- *Sonuç 2:* Kurulumdan sonra müşteri işlev isterse perfective maintenance; aynı değişikliği kurulumdan hemen önce isterse (**moving target problem**) klasik geliştirme sayılır.
- Ayrıca günümüzde sıfırdan geliştirme nadirdir, **yeniden kullanım (reuse)** yaygındır.

Modern tanım (ISO/IEC 12207, 1995; IEEE/EIA de kabul etti): Bakım, *bir yazılım ürünü bir problem nedeniyle ya da iyileştirme / uyarlama ihtiyacıyla değiştirildiğinde* gerçekleşen süreçtir. Yani **yazılım ne zaman değiştirilirse bakım vardır**; kurulumdan önce ya da sonra olması fark etmez. Bu **operational** (işleme dayalı) bir tanımdır.

- **Postdelivery maintenance:** teslim ve kurulumdan sonraki değişiklikler (IEEE 1990).
- **Modern maintenance** (ya da sadece maintenance): *her zaman* yapılan corrective, perfective veya adaptive bakım.

Sınav cevabı kalıbı

Classical maintenance is a temporal definition: any change after installation is maintenance (development-then-maintenance model). Modern maintenance (ISO/IEC 1995) is an operational definition: maintenance occurs whenever software is modified because of a problem or a need for improvement or adaptation, regardless of whether it takes place before or after installation.

1.3.4 Bakımın önemi

- Kötü yazılım atılır; **iyi yazılım 10–20 yıl bakımla yaşar**. Yazılım sürekli değişen gerçek dünyanın bir modelidir.
- 1976–1981 arası maliyetin yaklaşık **%67**'si, 1992–1998 arası **%75**'i (birçok kurumda %70–80) postdelivery maintenance'a gider.
- Geliştirme fazlarının kendi içindeki oranları ise neredeyse hiç değişmedi (gereksinim+analiz %18–21, tasarım %18–19, kodlama+birim testi %34–36, bütünleştirme %24–29).

1.4 Gereksinim, analiz ve tasarım boyutu

- **Bir hata ne kadar erken bulunursa o kadar ucuza düzeltilir.** Gereksinim fazındaki bir hatayı bakım aşamasında düzeltmek yaklaşık 100–200 kat pahalıdır (Boehm, logaritmik grafik).
- Erken düzeltmede genellikle sadece bir **doküman** değişir. Geç düzeltmede: kod ve doküman değişir, değişikliğin kendisi test edilir, **regression testing** yapılır, ürün müşterinin bilgisayar(lar)ına yeniden kurulur.
- Büyük projelerde hataların **%60–70**'i gereksinim, analiz ve tasarım hatasıdır. Jet Propulsion Laboratory incelemesi: spesifikasyonda sayfa başına 1,9; tasarımda 0,9; kodda 0,3 hata.
- Sonuç: gereksinim, analiz ve tasarım tekniklerini iyileştirmek hayati önemdedir.

1.5 Takım çalışması boyutu

Donanım ucuz olduğu için tek kişinin zamanında yazamayacağı kadar büyük ürünler yapılır; yazılımı **takımlar** geliştirir. Bunun iki sorunu vardır: modüller arası **arayüz (interfacing) problemleri** ve takım üyeleri arası **iletişim problemleri**.

1.6 Neden ayrı bir planlama, test ve dokümantasyon fazı yok?

Etkinlik	Neden ayrı faz değil?
Planning	Projenin başında neyin yapılacağını tam bilmediğimiz için baştan plan yapılamaz. Klasik paradigmada başta gereksinim ve analiz fazlarının ön planı yapılır; SPMP ancak müşteri spesifikasyonları onayladıktan (*signed off*) sonra hazırlanır ve proje boyunca izlenir. **Planlama bütün yaşam döngüsü boyunca yapılır.**
Testing	Geliştirme bittikten sonra, teslimden önce test etmek çok geçtir. **Verification** her fazın sonunda yapılan testtir (geç), **validation** projenin sonunda yapılan testtir (çok geç). Test sürekli yapılmalıdır; sorumlusu her yazılımcı ve **SQA (software quality assurance) grubu**dur.
Documentation	Sonradan yazmak çok geçtir: kilit kişiler belgeleme bitmeden ayrılabilir; önceki fazın belgesi olmadan sonraki faz yapılamaz; belge olmadan test ve bakım yapılamaz. Dokümantasyon **diğer bütün etkinliklerle paralel** yürütülür ve her zaman güncel olmalıdır.

Çıkmış test sorusu

"Which one is NOT a separate phase during software development?" Seçenekler: implementation, analysis, design, **documentation**, requirements. Cevap: **documentation**. Aynı mantıkla *testing* ve *planning* de ayrı faz değildir.

Verification ve validation farkı (sık sorulur): *verification* "ürünü doğru mu yapıyoruz?" (her fazın sonunda, o fazın doğru tamamlandığının kontrolü); *validation* "doğru ürünü mü yaptık?" (projenin sonunda ürünün spesifikasyonu karşıladığının kontrolü).

1.7 Nesne yönelimli paradigma

1.7.1 Yapısal paradigma neden yetersiz kaldı?

Yapısal (structured, classical) paradigma başta başarılıydı ama **50.000 satırdan büyük ürünlerde** başarısız olmaya başladı; bakım toplam eforun %70–80'ine çıktı. Sebep: yapısal yöntemler ya **action oriented** (sonlu durum makineleri, veri akış diyagramları) ya da **data oriented** (varlık-ilişki diyagramları, Jackson yöntemi) idi, **ikisi birden değil**.

1.7.2 Nesne

Nesne yönelimli paradigmada **veri ve eylemler eşit önemdedir**. **Nesne (object):** hem veriyi hem de o veri üzerinde yapılan eylemleri içeren yazılım bileşenidir. *Örnek: banka hesabı.* Veri: hesap bakiyesi (`accountBalance`). Eylemler: para yatır (`deposit`), para çek (`withdraw`), bakiyeyi öğren (`determineBalance`). Klasik versiyonda bakiyenin nasıl tutulduğunu bütün modüller bilir; nesne yönelimli versiyonda bakiye nesnenin içinde gizlidir ve dışarıdan yalnızca bu üç metot ile erişilir (**information hiding**).

1.7.3 Nesne yönelimli paradigmanın güçlü yanları

1. **Information hiding** sayesinde bakım daha güvenlidir: **regression fault** (görünüşte ilgisiz bir yerde ortaya çıkan hata) ihtimali azalır.
2. Geliştirme kolaydır: nesnelerin genellikle fiziksel karşılıkları vardır, bu da modellemeyi kolaylaştırır.
3. İyi tasarlanmış nesneler **bağımsız birimlerdir**: gerçek dünyadaki varlıkla ilgili her şey o nesnededir (**encapsulation**); iletişim **mesaj göndererek** olur; bağımsızlığı **responsibility-driven design** artırır.

4. Klasik ürün kavramsal olarak tek birimdir; nesne yönelimli ürün bağımsız birimlerden oluştuğu için **karmaşıklığı azaltır**.
5. Nesneler bağımsız olduğundan **yeniden kullanımı (reuse)** destekler.

1.7.4 Responsibility-driven design (design by contract)

Hocanın derste sorduğu soru: "Başka şehirdeki annene çiçek göndermek istiyorsun, ne yaparsın?" Çiçek sitesini (kitapta 1-800-flowers, derste Çiçeksepeti) ararsın. Çiçekçi nerede, hangi şehirdeki çiçekçi teslim edecek, bilmezsin; bilmen de gerekmez. **Bir nesnenin metoduna, nesnenin iç yapısını bilmeden mesaj gönderirsin**; işi yapmak o nesnenin **sorumluluğudur**. Bu, information hiding'in bir uygulamasıdır.

1.7.5 Analiz/tasarım "tümseği" (hump)

- Yapısal paradigmada analiz (*ne*) ile tasarım (*nasıl*) arasında sert bir geçiş (*jolt*) vardır: modüller ancak mimari tasarımda ortaya çıkar.
- Nesne yönelimli paradigmada nesneler **object-oriented analysis** iş akışında ortaya çıkar; tasarımda bu nesneler tasarlanır, gerçekleştirmede kodlanır. Geçişler yumuşaktır, "tümsek" kalkar.

1.7.6 Perspektif

Nesne yönelimli paradigma doğru kullanılırsa klasik paradigmanın *bazı* (hepsi değil) sorunlarını çözer; kendi sorunları da vardır. Bugün eldeki en iyi alternatiftir ama gelecekte daha iyisiyle yer değiştirecektir.

1.8 Terminoloji

Terim	Anlamı
Client / developer / user	Yazılımı isteyen / yapan / kullanan.
Internal software	Müşteri ve geliştirici aynı kurumda.
Contract software	Müşteri ve geliştirici farklı kurumlarda.
COTS (commercial off-the-shelf)	Hazır paket, raftan satılan ticari yazılım.
Open-source software	Gönüllülerce geliştirilir ve bakılır. *Linus's Law:* "Yeterince göz varsa bütün hatalar sığdır."
Program / system / product	Kod parçası / birlikte çalışan programlar / bunların tamamı ile belgeleri.
Methodology (paradigm)	Yazılım üretme yolu (OO paradigm, classical paradigm).
Technique	Bir etkinliği yürütme yöntemi.
Mistake → fault → failure → error	Programcının yaptığı yanlışlık (**mistake**) koda bir hata (**fault**) olarak girer; yazılım çalışınca bu hata gözlenen yanlış davranışa (**failure**) yol açar; **error**, sonucun *ne kadar yanlış olduğunun ölçüsüdür* (IEEE 610.12; hoca: "a quantitative definition"). **Defect**: fault, failure veya error için genel (şemsiye) terim. "Bug" terimi sorumluluğu atar ("koda bir böcek girdi" yerine "hata yaptım" denmeli).

Nesne yönelimli terimler: nesnenin veri bileşeni *state variable / instance variable (Java) / field (C++) / attribute (genel)*; eylem bileşeni *member function (C++) / method (genel).*

1.9 Etik konular (Ethical issues)

! Hoca derste vurguladı [video 2.4]

"i ask this code of ethics in the exam okay please study this code of ethics … there are eight principles." 2025 WhatsApp grubunda da kitabın ilgili sayfasına "Bazen sınavlarda soruyor" notu düşülmüş.

Yazılım geliştiren ve bakımını yapan kişilerin özellikleri: **hard working, intelligent, sensible, up to date** ve her şeyden önce **ethical**. (WhatsApp'ta bunlara *lifelong learner* da eklenmiş; bu, 8. ilkeden gelir.)

ACM ve IEEE-CS ortak olarak **Software Engineering Code of Ethics and Professional Practice** (sürüm 5.2, 1999) yayımladı. Kısa sürümü bir giriş (preamble) ve **sekiz ilkeden** oluşur:

#	İlke	Açıklama (İngilizce aslı ve Türkçe özü)
1	**Public**	*Software engineers shall act consistently with the public interest.* Kamu yararına uygun davran.
2	**Client and Employer**	*…act in a manner that is in the best interests of their client and employer consistent with the public interest.* Kamu yararıyla çelişmeden müşteri ve işverenin çıkarını gözet.
3	**Product**	*…ensure that their products and related modifications meet the highest professional standards possible.* Ürün ve değişikliklerde en yüksek profesyonel standart.
4	**Judgment**	*…maintain integrity and independence in their professional judgment.* Mesleki yargıda dürüstlük ve bağımsızlık.
5	**Management**	*Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.* Yöneticiler etik yönetimi benimser ve teşvik eder.
6	**Profession**	*…advance the integrity and reputation of the profession consistent with the public interest.* Mesleğin itibarını ve dürüstlüğünü yükselt.
7	**Colleagues**	*…be fair to and supportive of their colleagues.* Meslektaşlara adil ve destekleyici ol.
8	**Self**	*…participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.* Ömür boyu öğren, etik yaklaşımı teşvik et.

Ezber ipucu

Sıra: **P**ublic, **C**lient and Employer, **P**roduct, **J**udgment, **M**anagement, **P**rofession, **C**olleagues, **S**elf. Dışarıdan içeriye doğru düşünün: toplum → müşteri → ürün → kendi yargın → yönetim → meslek → meslektaşlar → kendin.

1.10 Kısa tanım soruları ("What is it?")

Software crisis Software is delivered late, over budget and with residual faults.

Regression fault A fault in an apparently unrelated part of the software, caused by a change made elsewhere. Information hiding reduces regression faults.

Regression testing Re-running all previous test cases after a change to check that nothing else is broken.

Moving target problem A change in the requirements while the software is being developed (Chapter 2).

Verification / validation Verification: testing at the end of each phase. Validation: testing at the end of the project (whether the product satisfies the specification).

Modern maintenance Maintenance occurs whenever software is modified, before or after installation.

Object A software component that incorporates both data and the actions performed on that data.

Mistake / fault / failure / error A mistake is made by a human; its consequence is a fault in the code; a failure is the observed incorrect behavior caused by the fault; the error is the amount by which a result is incorrect (IEEE 610.12).

Information hiding Hiding the implementation details of a module so that other modules cannot depend on them; future changes are localized.

Encapsulation Gathering everything that relates to a real-world item (data and actions) into one unit.

Responsibility-driven design Sending a message to an object without knowing how it performs the task (design by contract).